JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class IntSummaryStatistics

java.lang.Object
    java.util.IntSummaryStatistics

All Implemented Interfaces:

IntConsumer

```
public class IntSummaryStatistics
extends Object
implements IntConsumer
```

A state object for collecting statistics such as count, min, max, sum, and average.

This class is designed to work with (though does not require) streams. For example, you can compute summary statistics on a stream of ints with:

```
IntSummaryStatistics stats = intStream.collect(IntSummaryStatistics::new,
                                               IntSummaryStatistics::accept,
                                               IntSummaryStatistics::combine);
```

`IntSummaryStatistics` can be used as a reduction target for a stream. For example:

```
IntSummaryStatistics stats = people.stream()
                                   .collect(Collectors.summarizingInt(Person::ge
```

This computes, in a single pass, the count of people, as well as the minimum, maximum, sum, and average of their number of dependents.

Implementation Note:

This implementation is not thread safe. However, it is safe to use `Collectors.summarizingInt()` on a parallel stream, because the parallel implementation of `Stream.collect()` provides the necessary partitioning, isolation, and merging of results for safe and efficient parallel execution.

This implementation does not check for overflow of the count or the sum.

Since:

1.8

Constructor Summary

Constructors

Constructor	Description
`IntSummaryStatistics()`	Constructs an empty instance with zero count, zero sum, `Integer.MAX_VALUE` min, `Integer.MIN_VALUE` max and zero average.
`IntSummaryStatistics(long count, int min, int max, long sum)`	Constructs a non-empty instance with the specified `count`, `min`, `max`, and `sum`.

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
`void`	`accept(int value)`	Records a new value into the summary information
`void`	`combine (IntSummaryStatistics ot`	Combines the state of another `IntSummaryStatistics` into this one.
`final double`	`getAverage()`	Returns the arithmetic mean of values recorded, or zero if no values have been recorded.
`final long`	`getCount()`	Returns the count of values recorded.
`final int`	`getMax()`	Returns the maximum value recorded, or `Integer.MIN_VALUE` if no values have been recorded.
`final int`	`getMin()`	Returns the minimum value recorded, or `Integer.MAX_VALUE` if no values have been recorded.

final long	**getSum**()	Returns the sum of values recorded, or zero if no values have been recorded.
String	**toString**()	Returns a non-empty string representation of this object suitable for debugging.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Methods declared in interface IntConsumer

andThen

Constructor Details

IntSummaryStatistics

```
public IntSummaryStatistics()
```

Constructs an empty instance with zero count, zero sum, `Integer.MAX_VALUE` min, `Integer.MIN_VALUE` max and zero average.

IntSummaryStatistics

```
public IntSummaryStatistics(long count,
                            int min,
                            int max,
                            long sum)
                     throws IllegalArgumentException
```

Constructs a non-empty instance with the specified `count`, `min`, `max`, and `sum`.

If `count` is zero then the remaining arguments are ignored and an empty instance is constructed.

If the arguments are inconsistent then an `IllegalArgumentException` is thrown. The necessary consistent argument conditions are:

- `count >= 0`
- `min <= max`

API Note:

The enforcement of argument correctness means that the retrieved set of recorded values obtained from a `IntSummaryStatistics` source instance may not be a legal set of arguments for this constructor due to arithmetic overflow of the source's recorded count of values. The consistent argument conditions are not sufficient to prevent the creation of an internally inconsistent instance. An example of such a state would be an instance with: `count` = 2, `min` = 1, `max` = 2, and `sum` = 0.

Parameters:

`count` - the count of values

`min` - the minimum value

`max` - the maximum value

`sum` - the sum of all values

Throws:

`IllegalArgumentException` - if the arguments are inconsistent

Since:

10

Method Details

accept

```
public void accept(int value)
```

Records a new value into the summary information

Specified by:

`accept` in interface `IntConsumer`

Parameters:

`value` - the input value

combine

```
public void combine(IntSummaryStatistics other)
```

Combines the state of another `IntSummaryStatistics` into this one.

Parameters:

`other` - another `IntSummaryStatistics`

Throws:

`NullPointerException` - if `other` is null

getCount

```
public final long getCount()
```

Returns the count of values recorded.

Returns:

the count of values

getSum

```
public final long getSum()
```

Returns the sum of values recorded, or zero if no values have been recorded.

Returns:

the sum of values, or zero if none

getMin

```
public final int getMin()
```

Returns the minimum value recorded, or `Integer.MAX_VALUE` if no values have been recorded.

Returns:

the minimum value, or `Integer.MAX_VALUE` if none

getMax

```
public final int getMax()
```

Returns the maximum value recorded, or `Integer.MIN_VALUE` if no values have been recorded.

Returns:

the maximum value, or `Integer.MIN_VALUE` if none

getAverage

```
public final double getAverage()
```

Returns the arithmetic mean of values recorded, or zero if no values have been recorded.

Returns:

the arithmetic mean of values, or zero if none

toString

`public String toString()`

Returns a non-empty string representation of this object suitable for debugging. The exact presentation format is unspecified and may vary between implementations and versions.

Overrides:

`toString` in class `Object`

Returns:

a string representation of the object

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.